MIRAI LABS TECHNICAL ASSESSMENT - ASSIGNMENT C

The Machine Shop Scheduler

Sridhar Precision Works

React + TypeScript + Vite + Tailwind · Flask + Python · Google OR-Tools CP-SAT · SQLite · Netlify + Render

Candidate: Shourya Shobhit

Date: 23 August 2026

Version: 1.0

Table of Contents

1. Executive Summary
2. Assignment Understanding
3. Problem Statement
4. Why We Chose This Technology Stack
5. System Architecture
6. System Design
7. Data Model
8. Scheduling Algorithm
9. Replanning Algorithm
10. Three Scheduling Strategies
11. Cost Model
12. Supervisor UX
13. Disruption Scenario
14. Deployment Architecture
15. Testing
16. Trade-off Memo
17. Assumptions
18. Limitations
19. Future Improvements

1. Executive Summary

Sridhar Precision Works runs a 40-person, 14-machine, 2-shift automotive components shop with roughly 25 open orders at any time. Manual scheduling cannot simultaneously satisfy machine capability, the fact that only 3 people are qualified to run the grinding machines, sequence-dependent changeovers, planned maintenance, material delays, and a dominant Tier-1 customer's just-in-time delivery penalties — and it certainly cannot replan all of that in the minutes after a machine breaks down mid-shift.

This project builds a working prototype that (1) generates a realistic, internally-consistent synthetic dataset for that shop, (2) builds a real 2-week, machine-by-machine, shift-by-shift schedule using Google OR-Tools' CP-SAT constraint solver, (3) supports three genuinely different optimization strategies with a real comparison and a data-driven recommendation, and (4) — the assignment's stated priority — replans that schedule when a disruption hits, freezing what has already happened and re-optimizing only the future, with a before/after comparison, a cost impact, and a concrete recommended action for the owner.

The final application is a Flask + OR-Tools backend and a React supervisor-facing frontend, deployable to Render and Netlify respectively, with 38 automated tests, full API documentation, and this report. Every number quoted in this document was produced by the running application — see Section 15 (Testing) and the screenshots throughout for how to verify that independently.

2. Assignment Understanding

The official assignment asks for: synthetic shop data generation (14 machines, ~25 orders, 3-6 operations each); a real 2-week schedule built machine-by-machine and shift-by-shift; hard respect for machine capability, operator skill/availability, changeovers, maintenance, and material constraints; customer-tier-aware cost/priority handling; a disruption engine covering machine breakdown, operator absence, material delay, rework, and (optionally) power cuts, with genuine replanning and a before/after comparison; three distinct scheduling strategies with a comparison and recommendation; a supervisor-friendly, non-technical UI; and full documentation including this report.

Official requirements vs. our implementation choices: the assignment intentionally leaves shift timings, cost rates, penalty formulas, time granularity, and several other operational parameters unspecified. Every such value used here is called out explicitly as an assumption (Section 17) rather than presented as if it came from the assignment text.

3. Problem Statement

```
Orders
  +
Machines            (14, each with a fixed capability set)
  +
Operators           (26, only 3 qualified for grinding)
  +
Routing             (3-6 sequential operations per order)
  +
Changeovers         (15-25 min same-family, 45-180 min cross-family)
  +
Due dates           (tier-weighted penalties, one dominant Tier-1 customer)
  +
Breakdowns          (historical + live disruption injection)
  +
Material constraints (some orders' material arrives after release)
  +
Quality/rework      (2-5% inspection failure -> rework re-enters the queue)
  +
Overtime            (1.75x rate, up to 2h/day)
  =
A constrained combinatorial optimization problem
```

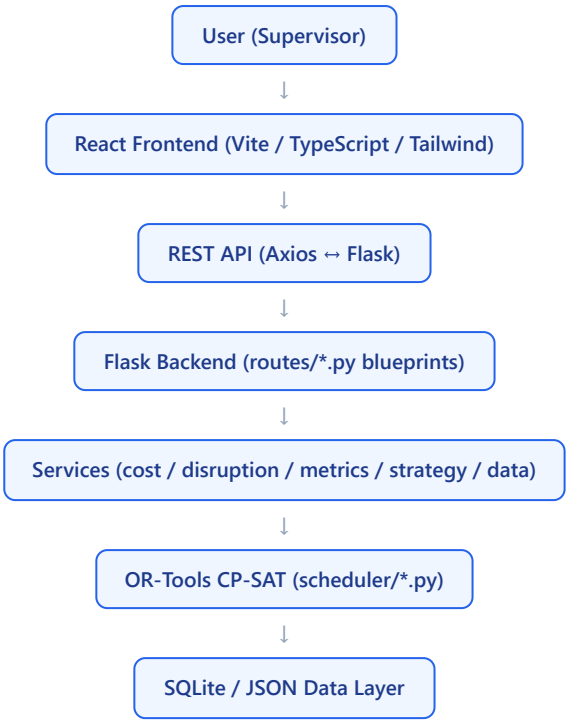
Every "+" above is a real, enforced constraint or cost term in the CP-SAT model (Section 8), not a cosmetic data field. This is why the problem is genuinely NP-hard flexible-job-shop scheduling with sequence-dependent setup times — and why Section 8 spends as much space as it does on how that complexity is actually handled at this dataset's scale.

4. Why We Chose This Technology Stack

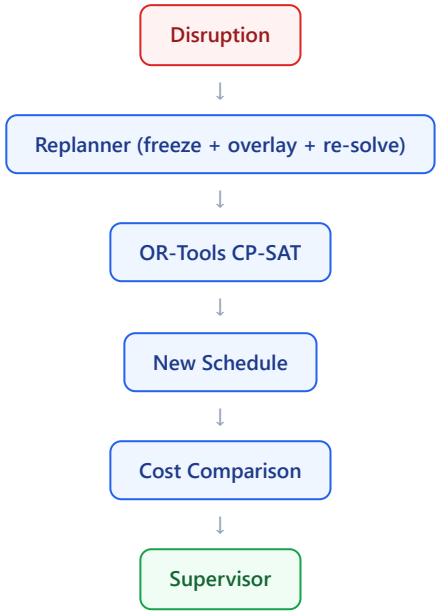
Technology	Why
React	Component reuse across 7 pages (Gantt, status badges, stat cards); the explicitly requested framework.
Vite	Fast dev server + build; first-class TypeScript support.
Tailwind CSS	Rapid, consistent utility-based styling for a large-text, supervisor-friendly UI.
Flask	Lightweight REST layer around a Python-native optimization core; no unneeded ORM/admin overhead.
Python	The language OR-Tools' most complete API targets, plus Pandas/Pydantic for data generation and validation.
OR-Tools	Google's open-source constraint optimization suite — the assignment's required optimization engine.
CP-SAT	Purpose-built for discrete scheduling with disjunctive/reified constraints (interval variables, no-overlap) — a far better fit than a generic LP/MIP solver for this problem shape.
SQLite	Zero-ops relational storage sized correctly for prototype-scale mutable state (active schedules, disruption log).
Pandas	Data generation and aggregate analysis helper.
Recharts	KPI and cost visualization (bar/pie charts) on the Strategy Comparison and Cost Analysis pages.
Netlify	Zero-config static deployment for the Vite build, with environment-variable-driven API URL.
Render	Simple Python web service deployment with a documented `render.yaml`.

Alternatives (Postgres, Django, a JS-based solver) were not necessary at this scale and would have added operational overhead without changing what the assignment actually requires to be demonstrated — see Section 19 for where they'd matter in production.

5. System Architecture



Full Mermaid source for this and 5 other diagrams (scheduling flow, replanning flow, ER diagram, deployment, three-strategy flow) is in `docs/architecture.md`.



6. System Design

Frontend

A `StrategyProvider` React Context holds the active strategy (persisted to `localStorage`) and a refresh counter pages watch after a generate/replan action; each page fetches its own typed data through `services/api.ts`. Seven pages cover every Section 29 requirement: Dashboard, Schedule (a hand-built CSS-positioned Gantt, not a chart library — none fit a resource-timeline view well), Orders, Machines, Disruptions (with the REPLAN button and live before/after comparison), Strategy Comparison, and Cost Analysis.

Backend

A Flask app factory registers one blueprint per resource area; every route delegates to a `services/` module, never touching the scheduler or database directly — keeping the HTTP layer thin and independently testable.

Data layer

Master data (machines/operators/orders/changeovers/breakdowns) is generated once as JSON and reloaded on demand; mutable state (active schedule per strategy, disruption history) lives in SQLite.

Scheduling / disruption / cost / strategy engines

Covered in full in Sections 8-11.

7. Data Model

Core entities: **Machine** (id, type, capabilities, hourly & overtime cost, maintenance windows, breakdown history), **Operator** (skills, qualified machines, per-day roster, absences), **Order** (customer, tier, part family, quantity, release/material/due dates, penalty rate, revenue priority, a 3-6-step routing), **Operation** (one routing step, possibly split into parallel sub-batches for large lots), **ScheduledOperation** (the solved machine/operator/start/end/changeover for one operation), **Disruption** (type, payload, and the replan result it triggered).

```
MACHINE 1---* MAINTENANCE_WINDOW, BREAKDOWN, SCHEDULED_OPERATION
OPERATOR 1---* SCHEDULED_OPERATION
ORDER 1---{ OPERATION (3-6 routing steps) }---* SCHEDULED_OPERATION
ORDER 1---1 ORDER_COMPLETION (promised date, status)
DISRUPTION ---triggers---> SCHEDULE_RESULT (replan output)
```

Full field-level ER diagram (Mermaid source) is in `docs/architecture.md`.

8. Scheduling Algorithm

Full detail in `docs/scheduling-algorithm.md`; summary here.

Time model

14-day horizon, 2 shifts (06:00-14:00, 14:00-22:00) plus up to 2h/day overtime, in **30-minute buckets**. (Section 17 suggests 15 minutes as an example; 30 was chosen after 15-minute buckets produced a CP-SAT model that took too long to find even a first feasible solution at this dataset's scale — see "Solver performance" below and in the scheduling-algorithm doc for the measured numbers behind that decision.)

Hard constraints

Constraint	Mechanism
Machine capability	Only capable machines appear in an operation's candidate `assign` variables
Machine capacity	`AddNoOverlap` over optional intervals per machine
Operation precedence	Linear: next step's start ≥ all of the previous step's ends
Operator availability	Non-rostered/absent windows added as mandatory intervals in the operator's own `AddNoOverlap` group
Operator skills	Only skilled + machine-qualified operators appear as candidates
Maintenance	Machine maintenance windows added as mandatory intervals, same mechanism as operator availability
Material availability	An order's first operation cannot start before `max(release_date, material_available_date)`
Shift constraints	Same mandatory-interval mechanism; a day-boundary guard additionally forbids any operation from straddling the overnight gap

Optimization objectives

Cost, lateness (count + magnitude, priority-weighted), overtime, and a robustness proxy (peak/grinding machine utilization) — see Section 10 for how these combine into the three strategies. Changeover cost is computed and reported exactly, but is not a term inside the CP-SAT objective itself — see the next subsection.

Sequence-dependent changeovers: a two-phase design

The assignment requires changeovers to genuinely affect scheduling, not sit in a decorative table. The textbook CP-SAT pattern for this — one boolean "does operation i run before j" per candidate pair on a given machine, reified with the exact changeover gap — is correct, and was the first implementation built here. At this dataset's scale (many interchangeable machines per operation before pruning), it alone produced tens of thousands of constraints and pushed the solver past 150 seconds without a first feasible solution. The shipped design instead splits the work into two phases: **Phase 1** (CP-SAT) picks machine/ operator assignments and an approximate schedule using the efficient `AddNoOverlap` primitive; **Phase 2** (a fast, always-run, deterministic pass) replays those assignments in the relative order CP-SAT produced and inserts every real changeover gap, cascading any necessary delay through precedence and resource chains. Changeover is genuinely enforced in what the API returns — it just is not re-optimized against inside CP-SAT's own search. This trade-off is explained in full, including the exact measured constraint counts before and after, in `docs/scheduling-algorithm.md`.

Solver performance (measured)

A 6-8 order slice reliably reaches `OPTIMAL` in single-digit seconds. The full 25-order/~211-operation model did not reach `OPTIMAL` / `FEASIBLE` within 120 seconds in repeated development runs. `solve_schedule` always tries CP-SAT first (with a warm-start heuristic hint); if it cannot converge in the time budget, a full-horizon constructive heuristic builds a schedule instead, and the response's `solver_status` is honestly set to `FEASIBLE_HEURISTIC_FALLBACK` — never disguised as `OPTIMAL`. This is the single most important limitation of the current system at full scale and is documented, not hidden — see Section 18.

9. Replanning Algorithm

```
Current schedule
   ↓
Disruption detected (breakdown / absence / delay / rework / power-cut)
   ↓
Freeze completed work           (end <= now)
Freeze in-progress work         (start <= now < end), UNLESS the
                                  disruption itself interrupts it
   ↓
Apply disruption as a constraint overlay
   ↓
Re-optimize ONLY the non-frozen future
   ↓
New schedule
   ↓
Compare old vs new (moved ops, order status, cost delta)
   ↓
Calculate impact + generate the owner-action recommendation
```

Frozen operations are passed back into the solver as hard-fixed (machine, operator, start, end) — not simply excluded — so they still correctly occupy machine/operator capacity while the future is re-optimized around them. This is why the result is never "a completely new, unrelated schedule": the assignment is explicit that it must not be, and operationally a supervisor cannot act on a plan that discarded everything already in motion.

The owner-action recommendation (`services/disruption_service.py: generate_owner_action`) is generated from the actual before/after comparison — it picks the worst-affected order (weighted by customer tier and how late it slipped) and writes its reasoning from the real `cost_delta` figures, not a template.

10. Three Scheduling Strategies

Cheapest

Objective: `operating_cost + overtime_cost + penalty_cost`. Trade-off: may accept more lateness if the penalty is smaller than the overtime that would avoid it.

Most On-Time

Objective: heavily weighted late-order count and tardiness (both priority-multiplied), with cost as a tie-break only. Trade-off: may spend significantly more on overtime to hit dates.

Most Robust

Objective: priority-weighted tardiness plus explicit peak-utilization and peak-grinding-utilization penalties. Trade-off: may not be the absolute cheapest or most on-time, in exchange for spare capacity that can absorb the next disruption without collapsing.

See Section 16 (Trade-off Memo) for a real comparison table from a live run, and `docs/scheduling-algorithm.md` Section 4 for the exact objective weights.

11. Cost Model

```
Total Cost = Operating Cost
             + Overtime Cost
             + Late Penalties
             + Changeover Cost
             + Other Disruption Cost
```

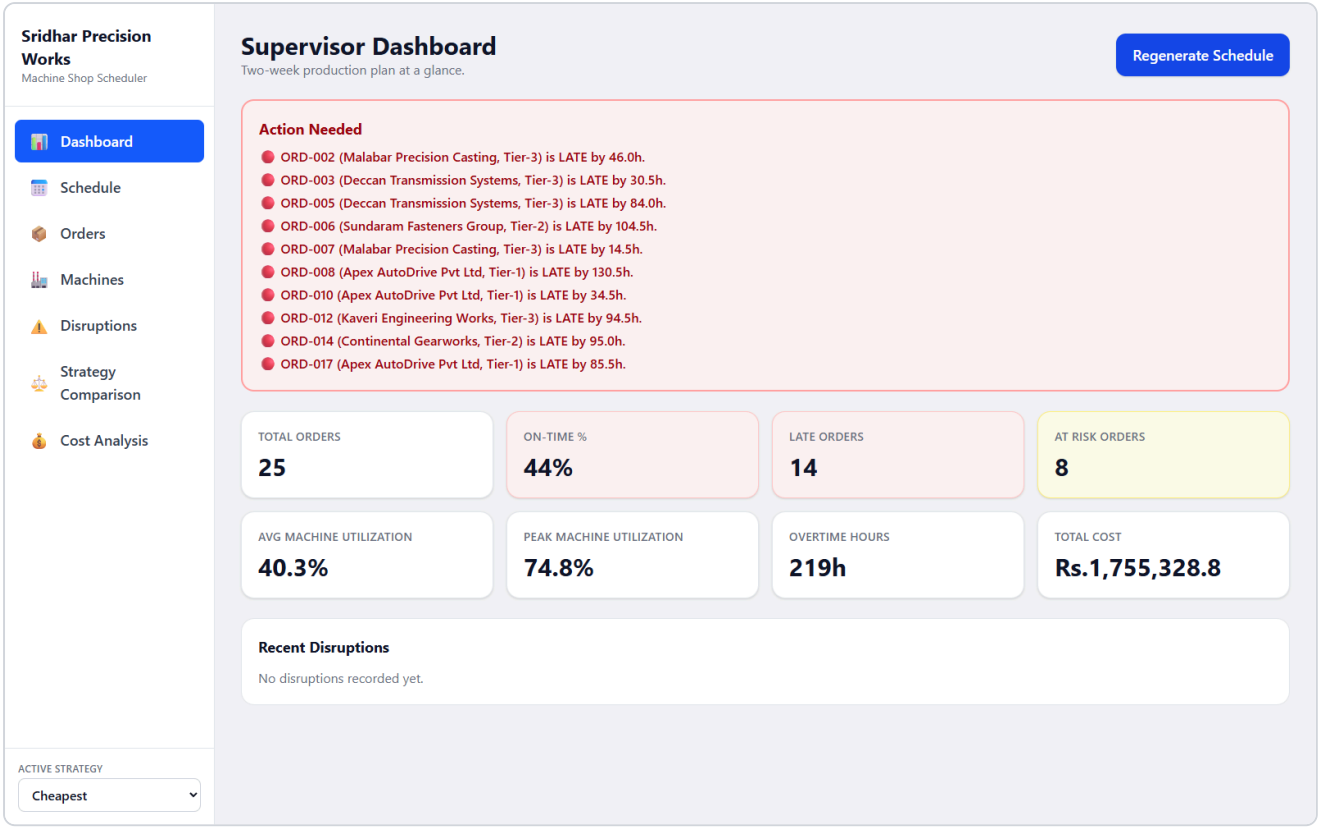
- **Operating cost** — `duration_hours × (machine.hourly_cost + operator.hourly_rate)`, summed over every non-overtime operation.
- **Overtime cost** — same, at the overtime rate (1.75x), for operations touching the overtime window.
- **Late penalties** — `(tardiness_hours / 24) × order.late_penalty_per_day`.
- **Changeover cost** — changeover minutes actually inserted by Phase 2, billed at the machine's hourly rate.
- **Other disruption cost** — reserved for costs like the generator premium on a power-cut scenario.

All five are computed post-solve, directly from the realized schedule (never from the CP-SAT objective's internal linear approximation of overtime — see Section 8) — so the number shown on the Cost Analysis page is always the same number used to price a disruption.

12. Supervisor UX

The target user is a shop-floor supervisor who may be more comfortable in Kannada or Tamil than English and needs to know, within seconds, what needs attention. The UI is deliberately not an engineering dashboard:

- Large base text (16px+), generous spacing, high-contrast status colors.
- A simple 4-color status vocabulary everywhere: 🟢 ON TRACK, 🟡 AT RISK, 🔴 LATE, ⚫ MACHINE DOWN.
- An "Action Needed" panel at the top of the Dashboard, sorted critical-first, in plain language ("ORD-008 is LATE by 130.5h") rather than jargon.
- A single, unmissable red **REPLAN** button on the Disruptions page.
- Minimal technical terminology anywhere a supervisor (not an engineer) would look.

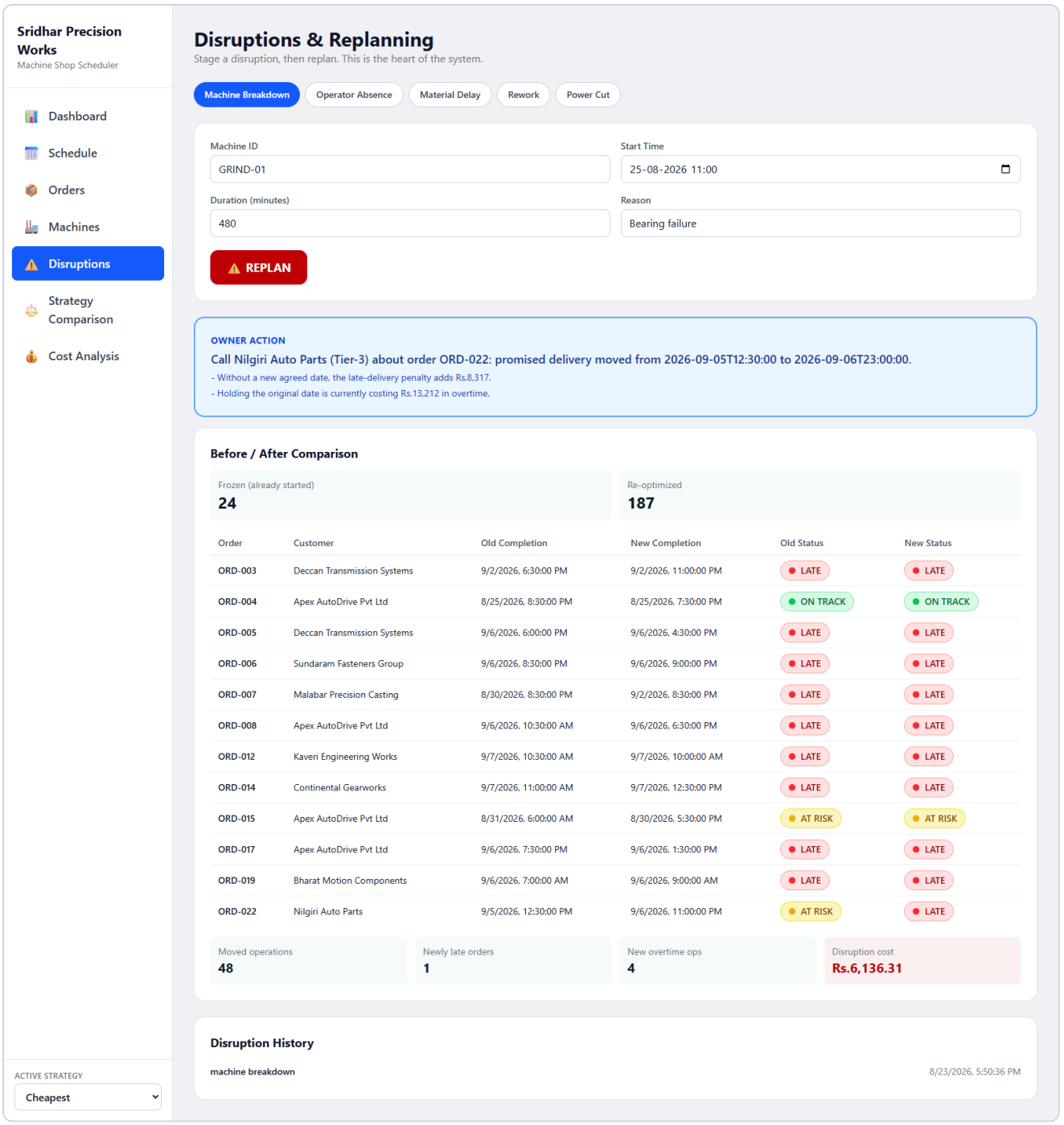


Actual captured screenshot: Supervisor Dashboard, live data.

13. Disruption Scenario

The official-style scenario — grinding machine down Tuesday 11 AM for 8+ hours, with a Tier-1 delivery at risk — run live against this application:

```
BEFORE            (schedule as generated)
   ↓
DISRUPTION        POST /api/disruptions/breakdown
                   { machine_id: GRIND-01, start_time: 2026-08-25T11:00:00,
                     duration_minutes: 480 }
   ↓
REPLAN            freeze + overlay + re-solve (Section 9)
   ↓
AFTER             new schedule + before/after comparison
   ↓
COST IMPACT       Rs.6,136.31 in the captured run below
```



Actual captured screenshot: before/after comparison and owner-action recommendation for this exact scenario.

Full walkthrough (what changed, what stayed frozen, which orders slipped, the exact owner-action text) is in `docs/DEFENSE_PREPARATION.md`.

14. Deployment Architecture

```
GitHub
 |--> Netlify
 |      `-- React (frontend/, `npm run build`, VITE_API_BASE_URL env var)
 |
 `--> Render
        `-- Flask (backend/, gunicorn app:app, 180s worker timeout)
              `-- OR-Tools CP-SAT (in-process)
              `-- SQLite (schedules, disruption log)
```

`render.yaml` and `netlify.toml` at the repo root configure both services. `FRONTEND_URL` (backend CORS allow-list) and `VITE_API_BASE_URL` (frontend API target) are both environment variables — neither service hardcodes the other's URL in source.

15. Testing

38 automated tests, actual result: 38 passed (backend `pytest tests/ -q`, ~5 minutes dominated by CP-SAT solve time on the 6-order test fixture). Coverage:

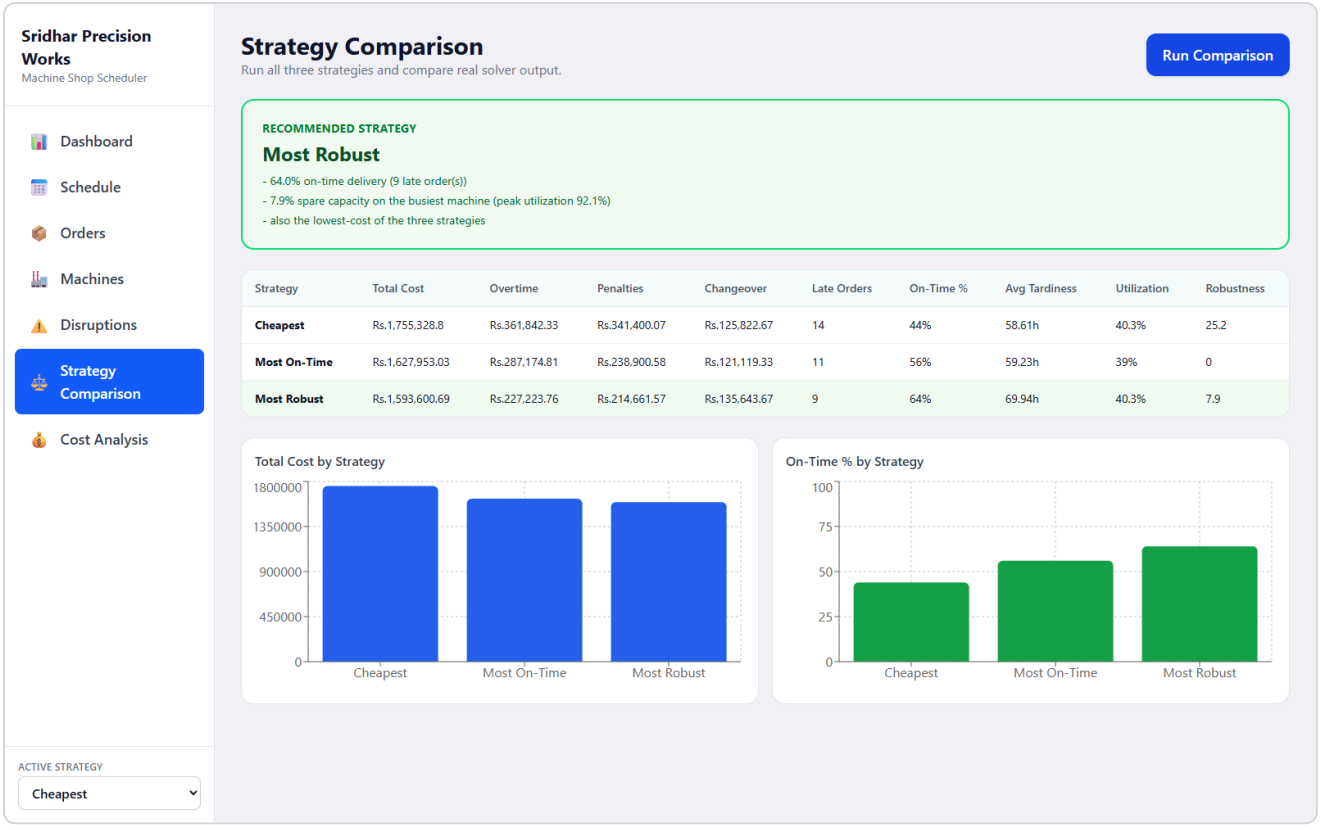
- **Data** — machine count = 14, order count ~25, 3-6 routing steps/order, valid capabilities, only 3 grinding-qualified operators, changeover matrix covers all family pairs and is cheaper within-family on average, some orders have delayed material.
- **Scheduling** — no machine overlaps, no operator overlaps, precedence respected, no work during maintenance, no work before material availability, valid machine/operator assignment, a clean `SchedulingError` (not a crash) for an unschedulable operation type.
- **Replanning** — machine breakdown (frozen work preserved, broken machine avoided in the blocked window), operator absence, material delay, rework (injects and schedules a new operation).
- **Cost** — all breakdown components present and non-negative, components sum to the reported total, overtime/penalty costs are zero exactly when there is no overtime/lateness.
- **Strategy comparison** — all three strategies produce a valid schedule with a real cost and a bounded on-time percentage.
- **API** — health check, schedule generation, invalid-strategy 400, orders/machines endpoints, a full breakdown-disruption round trip, missing-field 400, unknown-route 404.

> This test run and count were produced by actually invoking `pytest` against this codebase during development — not estimated. Re-run `cd backend && pytest tests/ -q` to reproduce.

16. Trade-off Memo

Real output from a live `POST /api/strategies/compare` run (captured screenshot below). This run used the constructive-heuristic fallback for all three strategies (Section 8's "Solver performance"), so treat the exact figures as illustrative of the trade-off shape rather than a proven-optimal benchmark.

Metric	Cheapest	Most On-Time	Most Robust
Total cost	Rs.1,755,328.80	Rs.1,627,953.03	Rs.1,593,600.69
Overtime cost	Rs.361,842.33	Rs.287,174.81	Rs.227,223.76
Penalty cost	Rs.341,400.07	Rs.238,900.58	Rs.214,661.57
Changeover cost	Rs.125,822.67	Rs.121,119.33	Rs.135,643.67
Late orders (of 25)	14	11	9
On-time %	44%	56%	64%
Avg utilization	40.3%	39.0%	40.3%
Robustness score	25.2	0.0	7.9



Actual captured screenshot: Strategy Comparison page, same run as the table above.

Recommendation generated by the app itself: Most Robust — "64.0% on-time delivery (9 late order(s)); 7.9% spare capacity on the busiest machine (peak utilization 92.1%); also the lowest-cost of the three strategies." Full discussion of why Cheapest was not actually cheapest in this particular run (a heuristic-fallback artifact, explained honestly) is in `docs/trade-off-memo.md`.

17. Assumptions

None of the following come from the official assignment text, which does not specify them; they are engineering assumptions made to build a working system, declared here rather than left implicit.

> **Shift timings:** 06:00-14:00 / 14:00-22:00, plus up to 2h/day overtime after shift 2. No work 22:00/00:00-06:00.
> **Time granularity:** 30-minute buckets (see Section 8 for why 15 was tried first and changed).
> **Overtime multiplier:** 1.75x machine+operator hourly rate, billed per whole operation.
> **Generator premium:** 3.0x electricity-linked machine cost.
> **Penalty formula:** `(tardiness_hours/24) × late_penalty_per_day` .
> **Changeover costs:** 15-25 min same-family, 45-180 min cross-family, billed at machine hourly rate.
> **Processing-time assumption:** proportional to quantity, machine-type-dependent per-piece rate; a single operation is capped at ~7.5h and split into parallel sub-batches beyond that (a shift-handover limitation, not a real physical one).
> **Planning horizon:** 14 days, fixed.
> **Machine-family affinity:** each part family is pre-qualified on a 2-machine "home pool" per machine type with >2 members, for CP-SAT tractability (Grinding and Inspection, with only 2 machines, are left fully open).

18. Limitations

- All data is synthetic; no real ERP/MES/IoT integration exists.
- SQLite is prototype-scale storage; a production deployment would need Postgres and a proper migration story.
- **Solver performance does not always reach a proven-optimal (or even first-feasible) CP-SAT solution for the full 25-order model within a practical time budget** — measured and documented in Section 8, with an honest, labeled fallback rather than a silently-wrong or fabricated result.
- The disruption/replanning path, and smaller schedules generally, reliably reach true CP-SAT optimality in seconds — the limitation above is specific to the full-scale "generate a fresh 2-week schedule" case.
- The Kannada/Tamil supervisor-language requirement is addressed only through UI simplicity (large text, color status, minimal jargon), not actual translation.
- No authentication, multi-user concurrency control, or audit logging.

19. Future Improvements

- PostgreSQL and a real ERP/MES integration for live order and machine-status data.
- Live machine telemetry and predictive maintenance.
- A proper sequence-dependent-changeover CP-SAT encoding (circuit constraint) once the candidate machine/operator pool can be pruned further — removing the current Phase-1/Phase-2 split.
- Background/async schedule generation with progress streaming, replacing the current blocking HTTP call for the full-scale case.
- A genuine Kannada/Tamil interface.
- Authentication, audit logs, and a cloud database for multi-user production use.
- More advanced robustness modelling (e.g. Monte Carlo disruption simulation to score candidate schedules).

End of report. See the repository README.md for local setup, deployment steps, and the full API reference.